

Chapter 2 Exercises

xm 2.1 What is the purpose of system calls?

System calls provide the interface between a **user program** and the **operating system kernel**.

A program running in user mode cannot directly access hardware or critical system resources. So when it needs services like:

- Creating a file
- Reading or writing data
- Creating a new process
- Allocating memory
- Communicating with another process

it must request the operating system through **system calls**.

In short:

👉 **System calls allow user programs to request services from the operating system safely and in a controlled way.**

They act as a bridge between user space and kernel space.

2.2 What is the purpose of the command interpreter? Why is it usually separate from the kernel?

☒ Purpose of the Command Interpreter

The command interpreter, also called a **shell**, allows users to interact with the operating system.

Its main responsibilities are:

- Reading user commands
- Interpreting those commands
- Executing programs
- Managing processes

For example, in UNIX systems like Unix, common shells include Bash and Zsh.

So basically:

👉 The command interpreter provides the **user interface** to the operating system.

☑ Why is it separate from the kernel?

It is usually separate from the kernel because:

1. **Modularity** – The kernel stays small and focused on core tasks like memory management and process scheduling.
2. **Flexibility** – Users can choose different shells without modifying the kernel.
3. **Security & Stability** – If the shell crashes, the kernel continues running.
4. **Easier development** – New shells can be developed without changing kernel code.

So, keeping the shell separate makes the system more stable and flexible.

2.3 What system calls are executed by a command interpreter (shell) to start a new process in UNIX?

When you type a command in a UNIX shell, the following system calls are typically used:

1 **fork()**

- Creates a new child process.
- The new process is a copy of the parent (shell).

2 **exec()**

- Replaces the child process's memory with a new program.
- Loads and runs the requested program.

3 **wait()**

- The parent process (shell) waits for the child to finish (for foreground processes).

Execution Flow:

1. User types command.
2. Shell calls `fork()`.
3. Child process calls `exec()` to run the program.
4. Parent calls `wait()` (if needed).

So the main system calls are:
👉 **fork(), exec(), and wait()**

2.4 What is the purpose of system programs?

System programs provide a convenient environment for program development and execution.

They are built on top of system calls and make the system easier to use.

Examples include:

- File management programs (copy, delete, list files)
- File modification programs
- Status information programs (date, time, memory usage)
- Programming tools (compilers, debuggers)
- Communication programs
- Background services (daemons)

For example:

- GCC – compiler
- ls – list files
- top – process monitoring tool

In simple terms:

👉 **System programs provide useful utilities that make the operating system convenient and usable for users and developers.**

They are not part of the kernel, but they rely on it.

2.5 What is the main advantage of the layered approach to system design? What are the disadvantages of the layered approach?

☒ **Advantage:**

- The layered approach divides the operating system into **hierarchical layers**, where each layer provides services to the layer above and relies on the layer below.

- **Main advantage:**
Simplifies design, implementation, and debugging.
Each layer can be developed and tested independently, which improves modularity and system reliability.

☑ Disadvantages:

1. **Performance overhead:** Each layer adds extra processing; passing requests up and down layers can slow execution.
2. **Rigidity:** Layers must strictly interact only with adjacent layers, making some designs inefficient or inflexible.
3. **Complex layer design:** Some OS functions don't naturally fit into a strict layered structure.

xm 2.6 List five services provided by an operating system, and explain how each creates convenience for users. In which cases would it be impossible for user-level programs to provide these services?

Five OS services:

1. Program Execution

- OS loads programs into memory and executes them.
- **User convenience:** Users don't need to manage memory allocation or CPU scheduling manually.
- **Impossible for user-level programs:** Programs cannot directly access CPU or memory hardware safely.

2. File System Management

- OS provides a structured way to store, retrieve, and organize files.
- **User convenience:** Users can store and access data without worrying about physical disk details.
- **Impossible for user-level programs:** Direct disk access risks data corruption and security issues.

3. I/O Device Management

- OS handles devices like printers, keyboards, displays.
- **User convenience:** Users don't need to write device-specific code.
- **Impossible for user-level programs:** Hardware requires privileged instructions only the OS can execute.

4. Communication / Networking Services

- OS allows processes to communicate across computers or processes.
- **User convenience:** Users can share data or run distributed applications easily.

- **Impossible for user-level programs:** Direct network hardware access is restricted and unsafe.

5. Security and Protection

- **OS enforces access controls and prevents unauthorized access.**
 - **User convenience:** Users' data and programs are protected without manual intervention.
 - **Impossible for user-level programs:** They cannot enforce protection at the hardware level.
-

2.7 Why do some systems store the operating system in firmware, while others store it on disk?

- **Firmware storage (ROM / flash memory):**
 - OS is permanently stored and immediately available at startup.
 - **Advantages:** Fast boot, reliable, useful for embedded systems (e.g., routers, smartphones).
 - **Disadvantage:** Difficult to update frequently.
 - **Disk storage:**
 - **OS is loaded from disk into memory at boot time.**
 - **Advantages:** Easy to update or replace, supports multiple OS versions.
 - **Disadvantage:** Slightly slower boot process than firmware-based OS.
-

2.8 How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?

- **Designing a multi-boot system:**
 - **Install multiple operating systems on separate partitions or drives.**
 - Use a **boot loader** (like GRUB in Linux) that allows the user to select which OS to start.
 - **Bootstrap program tasks:**
 - Initialize hardware and memory.
 - Display a menu of available operating systems.
 - Load the selected OS kernel into memory.
 - Transfer control to the OS so it can begin execution.
-
-

2.9 The services and functions provided by an operating system can be divided into two main categories. Briefly describe the two categories, and discuss how they differ.

Two main categories:

1. User services (or convenience functions):

- These are services provided to **make the system convenient for users and application programs**.
- Examples: File management, program execution, user interface, networking utilities, text editors.
- Focus is on **ease of use** and productivity.

2. System services (or kernel-level functions):

- These are services that **manage and control hardware resources directly**.
- Examples: Process scheduling, memory management, I/O device control, security, interrupt handling.
- Focus is on **resource management and reliability**.

Difference:

- User services are designed **for user convenience** and run mostly in **user mode**.
 - System services **control hardware and core OS functionality** and run in **kernel mode**, providing safety and efficiency.
-

xm 2.10 Describe three general methods for passing parameters to the operating system.

1. Registers:

- Parameters are loaded into CPU registers before the system call.
- Efficient but limited by the number of registers.

2. Block or Table in Memory:

- Parameters are stored in a memory block, and a pointer to this block is passed to the OS.
- Useful for **passing large amounts of data**.

3. Stack:

- Parameters are pushed onto the process stack before the system call.
 - Common in high-level language calls and provides a flexible method for multiple parameters.
-

2.11 Describe how you could obtain a statistical profile of the amount of time a program spends executing different sections of its code. Discuss the importance of obtaining such a statistical profile.

Method: Profiling

1. Insert **timing or counting code** at the beginning and end of different code sections.
2. Use OS-provided **profiling tools** (like `gprof` in UNIX).
3. Record **number of instructions executed** or **time spent** in each function.

Importance:

- Identifies **performance bottlenecks**.
 - Helps optimize critical sections of code.
 - Improves CPU utilization and reduces execution time.
-

2.12 What are the advantages and disadvantages of using the same system call interface for manipulating both files and devices?

প্রশ্নটা হচ্ছে: একই system call interface ব্যবহার করে ফাইল এবং ডিভাইস দুইটিকে handle করার সুবিধা এবং অসুবিধা কী?

Advantages (সুবিধা)

1. **সহজ প্রোগ্রামিং**
 - একই function call দিয়ে ফাইল এবং ডিভাইস handle করা যায়।
 - যেমন: `read()` দিয়ে হঠাৎ CPU, disk বা keyboard থেকে data পড়তে পারি।
 2. **Consistency / Uniformity**
 - OS-এর interface standardized হয়। সব resource-এর জন্য একই ধরনের rules।
 3. **Code Reuse**
 - Programs সহজে পুনঃব্যবহারযোগ্য হয়। একবার `read()` function ব্যবহার শিখলে সব ধরনের input device এবং file-এর জন্য ব্যবহার করা যায়।
 4. **Less Learning Curve**
 - নতুন programmer সহজেই OS ব্যবহার শিখতে পারে, কারণ সব resource-এর জন্য একই interface।
-

Disadvantages (অসুবিধা)

1. **Device-Specific Features Handle করা কঠিন**
 - কিছু ডিভাইসের special feature থাকে যা সাধারণ file-এর মতো handle করা যায় না।
 - উদাহরণ: printer-এ print quality বা scanner-এ resolution adjust করা।
 2. **Performance Overhead**
 - Generic system call-এর কারণে কিছু devices-এর জন্য extra abstraction layer তৈরি হয়, যা execution ধীর করতে পারে।
 3. **Error Handling Complex**
 - Device-specific errors file-like error-এর মতো handle করতে গেলে অনেক সময় mismatch হয়।
-

2.13 Would it be possible for the user to develop a new command interpreter using the system-call interface provided by the operating system?

☒ Yes.

- A user can write a new shell by using system calls like `fork()`, `exec()`, `wait()`, and I/O calls.
 - The shell reads user commands, creates processes, executes programs, and manages input/output.
 - Example: UNIX shells like **Bash**, **Zsh**, or **Fish** were implemented entirely using the OS system-call interface.
-

2.14 Describe why Android uses ahead-of-time (AOT) rather than just-in-time (JIT) compilation.

- **AOT compilation** converts app code into native machine code **before execution**, while **JIT compilation** converts code at runtime.

Reasons Android uses AOT:

১. দ্রুত অ্যাপ লঞ্চ / Startup

- AOT ব্যবহার করলে অ্যাপের কোড **ইনস্টলেশনের সময়েই** native machine code-এ compile হয়ে যায়।
- এর ফলে অ্যাপ চালু করার সময় CPU সরাসরি কোড execute করতে পারে, কোনো runtime compilation-এর অপেক্ষা করতে হয় না।
- শুধুমাত্র JIT ব্যবহার করলে কোড runtime-এ compile করতে হয়, তাই অ্যাপ শুরুতে ধীর হয়ে যায়।

২. Better Runtime Performance

- AOT code সাধারণত **optimized** হয় hardware-এর জন্য।
- JIT code runtime-এ compile হওয়ার কারণে সবসময় maximum optimization করতে পারে না।

৩. কম ব্যাটারি খরচ

- JIT runtime-এ compile করার সময় **CPU বেশি কাজ করে**, যা ব্যাটারি খরচ বাড়ায়।
- AOT-এ **compile আগেই হয়ে গেলে** runtime CPU load কম থাকে।

৪. Memory Efficiency

- **Native code-এ convert হওয়ায় memory footprint কম থাকে**, কারণ runtime compilation-এর জন্য extra memory লাগেনা।

Trade-off:

- AOT increases **installation size** and **installation time**, but it improves runtime performance and battery efficiency.

2.15 What are the two models of interprocess communication? What are the strengths and weaknesses of the two approaches?

Two models of IPC (Interprocess Communication):

1. **Message Passing:**
 - Processes communicate by **sending and receiving messages via the OS.**
 - **Strengths:**
 - Works in distributed systems.
 - Simple synchronization (send/receive).
 - **Weaknesses:**

- Overhead due to context switching and copying messages.
- Slower for large data transfers.

2. Shared Memory:

- Processes communicate by reading and writing to a shared memory region.
- **Strengths:**
 - Fast communication since no message copying is needed.
 - Efficient for large data.
- **Weaknesses:**
 - Requires synchronization (mutexes, semaphores) to avoid conflicts.
 - More complex to program.

xm 2.16 Contrast and compare an application programming interface (API) and an application binary interface (ABI).

API (Application Programming Interface) হলো এমন একটি interface যা programmer ব্যবহার করে source code level-এ operating system বা library-এর সাথে কাজ করে।

ABI (Application Binary Interface) হলো এমন interface যা compiled program (machine code) এবং operating system-এর মধ্যে communication নির্ধারণ করে।

API এবং ABI-এর মিল (Similarities)

1. দুটোই program এবং OS বা library-এর মধ্যে communication এর জন্য ব্যবহৃত হয়।
2. দুটোই program execution-এর জন্য গুরুত্বপূর্ণ।
3. দুটোই interface হিসেবে কাজ করে।

API এবং ABI-এর পার্থক্য (Differences)

API	ABI
Source code level-এ কাজ করে	Binary level-এ কাজ করে
Programmer ব্যবহার করে	Compiler এবং OS ব্যবহার করে
Functions, syntax define করে	Machine level rules define করে
Example: read(), write()	Example: registers, memory calling convention

2.17 Why is the separation of mechanism and policy desirable?

Mechanism (মেকানিজম)

Mechanism হলো কিভাবে কাজটি হবে (how to do it) — অর্থাৎ implementation বা method।

👉 উদাহরণ: CPU scheduling করার system (queue, timer, context switch)

Policy (পলিসি)

Policy হলো কোন নিয়ম অনুযায়ী কাজটি হবে (what to do) — অর্থাৎ decision rule।

👉 উদাহরণ:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling

এগুলো decide করে কোন process আগে চলবে।

কেন Mechanism এবং Policy আলাদা রাখা ভালো

১. সহজে পরিবর্তন করা যায় (Flexibility)

- Policy change করা যায় mechanism না বদলেই।
 - যেমন: FCFS থেকে Priority scheduling এ switch করা যায়, mechanism একই রেখে।
-

২. System maintain করা সহজ (Easy maintenance)

- আলাদা থাকলে system update বা modify করা সহজ হয়।
 - পুরো system পরিবর্তন করতে হয় না।
-

৩. System বেশি portable হয় (Portability)

- একই mechanism ব্যবহার করে বিভিন্ন policy ব্যবহার করা যায়।
 - Different systems-এ easily adapt করা যায়।
-

8. Better system design (Modularity)

- System structured এবং organized থাকে।
 - Mechanism এবং policy independent হওয়ায় system clean থাকে।
-

৫. Security এবং control improve হয়

- OS easily control করতে পারে কোন policy ব্যবহার হবে।
-

xm 2.18 It is sometimes difficult to achieve a layered approach if two components of the operating system are dependent on each other. Identify a scenario in which it is unclear how to layer two system components that require tight coupling of their functionalities.

Layered approach কী (সংক্ষেপে)

Layered approach-এ OS-কে বিভিন্ন layer-এ ভাগ করা হয়, যেখানে

- নিচের layer → hardware-এর কাছাকাছি
- উপরের layer → user-এর কাছাকাছি
- প্রতিটি layer শুধু নিচের layer ব্যবহার করে

সমস্যা হয় যখন দুইটি component একে অপরকে simultaneously দরকার করে।

গুরুত্বপূর্ণ Scenario: Virtual Memory এবং File System

Virtual Memory কী করে

- RAM manage করে
- page ব্যবহার করে memory allocate করে
- page fault হলে disk থেকে data আনে

File System কী করে

- disk-এ file store এবং retrieve করে
- disk read/write করে

Problem কোথায় (Tight Coupling)

1. **Virtual Memory file system ব্যবহার করে**
 - যখন page fault হয়, virtual memory disk থেকে data আনে
 - disk access করতে file system দরকার
2. **File System virtual memory ব্যবহার করে**
 - file system data cache করার জন্য memory ব্যবহার করে
 - memory manage করতে virtual memory দরকার

👉 অর্থাৎ:

- Virtual memory → file system দরকার
- File system → virtual memory দরকার

এটা circular dependency তৈরি করে।

কেন layered approach এখানে unclear

Layered approach-এ normally একটি component নিচে এবং আরেকটি উপরে থাকবে। কিন্তু এখানে:

- Virtual memory কে নিচে রাখলে → file system কাজ করতে virtual memory দরকার
- File system কে নিচে রাখলে → virtual memory কাজ করতে file system দরকার

👉 তাই কোনটা নিচে আর কোনটা উপরে হবে, সেটা clear নয়।

2.19 What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

১. Microkernel approach-এর main advantage (মূল সুবিধা)

Microkernel-এর প্রধান সুবিধা হলো **modularity এবং reliability (নির্ভরযোগ্যতা)**।

ব্যাখ্যা:

- Microkernel-এ শুধুমাত্র essential services kernel-এর মধ্যে থাকে, যেমন:
 - Process management
 - Memory management
 - Interprocess Communication (IPC)
- অন্য services যেমন:
 - File system
 - Device drivers
 - Network services

এগুলো kernel-এর বাইরে user space-এ থাকে।

কেন এটা advantage:

- যদি কোনো service crash করে, পুরো OS crash করে না
- সহজে system update বা modify করা যায়
- system বেশি secure এবং stable হয়

২. User programs এবং system services কীভাবে interact করে (Interaction)

Microkernel architecture-এ communication হয় **message passing-এর মাধ্যমে**।

Step-by-step process:

1. User program request পাঠায় microkernel-এ
2. Microkernel সেই request message আকারে system service-এ পাঠায়
3. System service কাজ করে

4. Result আবার message আকারে microkernel-এর মাধ্যমে user program-এ ফিরে আসে

👉 এখানে microkernel একটি **mediator** (মধ্যস্থতাকারী) হিসেবে কাজ করে।

৩. Microkernel approach-এর disadvantages (অসুবিধা)

১. Performance কম হতে পারে

- বারবার message passing করতে হয়
- এতে overhead বাড়ে
- execution ধীর হতে পারে

২. Communication overhead বেশি

- kernel এবং user space-এর মধ্যে frequent communication হয়

৩. Implementation complex

- system design এবং communication mechanism জটিল হয়
-

2.20 What are the advantages of using loadable kernel modules?

Loadable Kernel Modules (LKM) কী?

Loadable Kernel Modules হলো kernel-এর এমন অংশ যেগুলো system চলাকালীন সময়ে dynamically load এবং unload করা যায়, kernel rebuild বা restart ছাড়াই।

উদাহরণ: device drivers, file system support ইত্যাদি।

Advantages (সুবিধা)

১. Kernel rebuild করার দরকার নেই

- নতুন feature বা driver add করতে পুরো kernel recompile করতে হয় না।
- শুধু module load করলেই কাজ হবে।

👉 এতে সময় এবং effort বাঁচে।

২. Memory efficient (মেমোরি সাশ্রয়ী)

- শুধু প্রয়োজনীয় module load করা হয়।
- প্রয়োজন না হলে unload করা যায়।

👉 এতে RAM কম ব্যবহার হয়।

৩. Easy to add new device support

- নতুন hardware (printer, USB, network card) support করার জন্য নতুন module load করলেই হয়।
 - পুরো OS পরিবর্তন করতে হয় না।
-

৪. System flexibility বৃদ্ধি পায়

- System easily modify এবং update করা যায়।
 - Different systems-এ different modules ব্যবহার করা যায়।
-

৫. System stability improve হয়

- কোনো module crash করলে পুরো kernel crash হওয়ার সম্ভাবনা কম।
 - Module unload করে problem fix করা যায়।
-

৬. Faster development and testing

- Developers সহজে module test করতে পারে kernel restart ছাড়াই।
- Debugging সহজ হয়।

2.21 How are iOS and Android similar? How are they different?

Similarities:

- Both are **mobile operating systems**.
- Provide **touch-based interfaces**, app frameworks, and security mechanisms.
- Both support multitasking and resource management.

Differences:

Aspect	iOS	Android
Kernel	XNU (hybrid)	Linux kernel
Programming language	Objective-C / Swift	Java / Kotlin
App distribution	App Store	Google Play (and others)
Customization	Limited	Highly customizable by OEMs and users

xm 2.22 Explain why Java programs running on Android systems do not use the standard Java API and virtual machine.

১. Mobile device-এর জন্য optimized virtual machine দরকার

Standard JVM মূলত desktop এবং server system-এর জন্য তৈরি করা হয়েছে (Oracle দ্বারা উন্নত)।

কিন্তু Android run করে mobile device-এ, যেখানে:

- RAM কম
- CPU power কম
- Battery limited

এই কারণে Google Android-এর জন্য special virtual machine তৈরি করেছে:

- আগে: Dalvik Virtual Machine (DVM)
- এখন: Android Runtime (ART)

এগুলো mobile device-এর জন্য optimized।

২. Different executable format ব্যবহার করে

Standard Java program compile হয়ে .class file তৈরি করে, যা JVM run করে।

কিন্তু Android program compile হয়ে .dex (Dalvik Executable) file তৈরি করে।

👉 এই .dex format memory efficient এবং mobile-friendly।

৩. Standard Java API পুরোটা mobile-এর জন্য suitable নয়

Standard Java API-তে অনেক feature আছে যেগুলো mobile device-এ দরকার নেই, যেমন:

- Desktop GUI (Swing, AWT)
- Heavy libraries

Android নিজস্ব API ব্যবহার করে, যা mobile features support করে:

- Touchscreen
 - Camera
 - GPS
 - Sensors
-

৪. Better performance এবং battery efficiency

Android-এর ART Ahead-of-Time (AOT) compilation ব্যবহার করে, ফলে:

- App faster run করে
- CPU usage কম হয়
- Battery কম খরচ হয়

Standard JVM-এ mostly Just-in-Time (JIT) compilation ব্যবহৃত হয়, যা mobile-এর জন্য কম efficient।

1.
